NATIONAL UNIVERSITY OF SINGAPORE



AY 2026/2027
EE3703 PROJECT PROPOSAL

Submitted By:

Du Ying Zeng Andrew A0264354A
Luo Ziyu A0351358B
Wang Shunyao A0355565U
Sun Zhiquan A0354826W
Guo Lexin A0356193X

NUS-ISS
INTELLIGENT REASONING SYSTEMS

IntelliHire AI — Intelligent Career Assessment & Interview System

An Integrated Platform for Resume-Based Job Recommendation, Digital-Human Adaptive Interviewing, and Evidence-Based Competency Assessment

Executive Summary

Problem. Candidates face two connected challenges: identifying roles that genuinely match their skills and preparing for role-specific interviews. Generic job browsing offers limited personalization, while generic mock interviews often ignore the candidate's actual projects. At the same time, highly adaptive interviewing can create inconsistent difficulty, uneven competency coverage, and opaque scoring.

Proposal. We propose an Intelligent Career Companion with three connected modules. First, a resume-driven recommendation engine ranks a team-maintained catalogue of computing roles and explains why each Top-K job matches the candidate. Second, the selected role configures an adaptive LLM interview that uses multi-source Retrieval-Augmented Generation (RAG), explicit interview state, and progressive probing. Third, a digital-human interface presents questions through text-to-speech, avatar lip-sync and basic gestures, captures spoken answers through automatic speech recognition, and passes only the transcript into the evidence-based evaluation pipeline.

Core contribution. A shared Candidate Profile connects job discovery and interview preparation: the same resume evidence used to recommend suitable roles also grounds role-specific interview planning. Personalization is allowed in both job ranking and question content, while assessment remains standardized. Different candidates may receive different recommendations and questions, but interview answers are mapped onto the same competency space and behaviorally anchored rating scale. The system reports score, assessment coverage, and confidence separately.

Proposal Structure

- 1. Introduction
- 2. Project Background / Market Context
- 3. Literature Review and Research Gap
- 4. Project Scope

- 5. Data Collection and Knowledge Preparation
- 6. System Design
- 7. Standardized Evaluation Mechanism
- 8. Evaluation Plan
- 9. Challenges and Risk Mitigation
- 10. Work Plan and Team Responsibilities
- 11. Expected Contributions and Conclusion
- References
- Appendix A. Responsible AI and Use of AI Tools

1. Introduction

Technical interviews are inherently heterogeneous. Candidates may have worked on LLM inference, recommendation systems, databases, computer vision, distributed systems, or research prototypes. Asking identical project questions would ignore this diversity, yet allowing each interview to evolve freely makes final scores difficult to compare. This creates a tension between personalization and standardization.

Our project addresses this tension with an intelligent reasoning system that adapts questions to each candidate while preserving a common evaluation framework. The interviewer reasons over an explicit interview state, including the active project, remaining time, competency coverage, evidence quality, question difficulty, and confidence. It then decides which competency to test next, retrieves relevant context, generates a targeted question, evaluates the resulting evidence, and updates its state before the next turn.

1.1 Objectives

- Build an explainable resume-to-job recommendation module that returns Top-K roles using semantic similarity, structured skill matching, experience fit, and optional user preferences.
- Create a digital-human interview layer using ASR, TTS, avatar lip-sync, and basic non-verbal animation while keeping visual and vocal traits outside the scoring pipeline.
- Create an adaptive interview orchestration mechanism that selects the next competency and question depth based on evidence coverage and confidence.
- Build a multi-source RAG layer over candidate documents, job requirements, technical knowledge, scoring rubrics, and question templates.
- Design a shared competency taxonomy and behaviorally anchored rating scale (BARS) for project-deep-dive interviews.
- Separate question generation from scoring so that evaluation is evidence-driven rather than based on overall LLM impression.
- Evaluate whether the proposed architecture improves cross-project consistency, human agreement, and score stability relative to direct LLM scoring.

1.2 Research Questions

1. RQ1: Can an AI interviewer generate project-specific follow-up questions that remain aligned with predefined technical competencies?
2. RQ2: Can heterogeneous answers be transformed into a common competency representation that supports cross-candidate comparison?
3. RQ3: Does evidence-based, rubric-driven scoring produce more stable and explainable judgments than direct transcript-to-score prompting?
4. RQ4: How does RAG affect question relevance, technical correctness, and interview depth?
5. RQ5: Can resume-derived semantic and skill features produce relevant, explainable Top-K job recommendations for a new user with no behavioral history?
6. RQ6: Can a digital-human interaction layer improve interview realism and usability while preserving the same transcript-based scoring logic?

2. Project Background / Market Context

Employment interviews remain one of the most widely used selection methods. Decades of research show that adding structure—standardized constructs, questions, probing rules, and scoring guidance—generally improves the psychometric properties of interviews [1], [2]. At the same time, technical hiring often depends on project-deep-dive discussion, where rigidly identical questions are not realistic because candidate experiences differ substantially.

AI-based recruitment tools are increasingly used across sourcing, screening, assessment, and interview workflows. This expansion creates opportunities for scalable interviewing, but also introduces concerns around bias, transparency, and the use of inappropriate proxy signals in candidate evaluation [9]. For this reason, our system deliberately avoids facial-expression scoring, accent analysis, personality inference from appearance, or fully autonomous hiring decisions. The prototype focuses on text-based technical evidence and is designed as a decision-support system.

2.1 The Core Problem

Table 1. Interview design trade-off

Interview approach	Strength	Limitation
Highly standardized interview	Comparable questions and scores	Poor fit for heterogeneous technical projects
Fully adaptive human/LLM interview	Rich project-specific probing	Uneven difficulty and coverage; hard to compare
Proposed hybrid approach	Adaptive content + common competency rubric	Requires explicit reasoning, calibration, and evidence tracking

The proposed research challenge is therefore not simply “how to generate interview questions.” It is how to make an adaptive interview behave like an active measurement process: the system must reason about what evidence is missing, what depth has been demonstrated, whether a new probe is justified, and how strongly a conclusion is supported.

3. Literature Review and Research Gap

3.1 Structured Interviews and Standardized Measurement

Campion, Palmer, and Campion reviewed multiple mechanisms for structuring selection interviews and concluded that interview structure improves important psychometric properties [1]. Levashina et al. later synthesized two decades of structured interview research, including rating-scale development, probing and follow-up, response elaboration, and bias reduction [2]. These findings motivate our decision to keep competencies, scoring anchors, and evidence requirements standardized even when project-specific questions are adaptive.

3.2 Behaviorally Anchored Rating Scales (BARS)

A generic 1–5 score is ambiguous unless raters share a common interpretation. Kell et al. describe behaviorally anchored rating scales as an important component of structured interviews and note their association with reliability, predictive validity, and reduced bias [3]. We therefore define each competency level using observable behaviors. For example, Technical Depth progresses from terminology recall to implementation knowledge, mechanism explanation, trade-off reasoning, and finally counterfactual reasoning under unseen constraints.

3.3 Retrieval-Augmented Generation and Agentic Reasoning

RAG combines parametric language-model knowledge with explicit non-parametric retrieval, improving access to external knowledge and supporting more grounded generation [4]. In our design, retrieval is separated into candidate, job, technical, rubric, and question collections so that agents receive task-specific context rather than a single undifferentiated prompt. ReAct further motivates an architecture in which reasoning and action are interleaved: an agent can update a plan, retrieve information, take an action, observe the result, and continue reasoning [5]. Surveys of LLM-agent planning similarly emphasize task decomposition, plan selection, external modules, reflection, and memory as core planning patterns [6].

3.4 Limitations of LLM-as-a-Judge

Directly asking an LLM to read an interview transcript and output a score is attractive but fragile. Wang et al. show that LLM evaluators can exhibit systematic position bias and propose calibration methods based on multiple evidence, balanced order, and human review [7]. Shi et al. further demonstrate that position bias varies across judges and tasks [8], while broader surveys of LLM-as-a-Judge identify reliability and bias as central challenges [10]. Our design therefore separates question generation from evaluation and requires explicit evidence extraction before scoring.

3.5 Resume–Job Matching and Recommendation

Job recommender systems commonly combine content, knowledge, behavioural, and hybrid signals, while new users create a cold-start challenge because no interaction history is available [19].
Resume–job matching research also shows that transformer models can jointly represent resume and

job-description context for suitability prediction [20]. These findings motivate a content-based first release: the system uses resume-derived skills, experience, projects, semantic embeddings, and explicit user preferences to rank a curated catalogue of roles. The recommendation remains explainable by exposing the main matched and missing skills rather than returning only an opaque similarity score.

3.6 Virtual Interviewers and Digital Humans

Prior work has demonstrated multimodal virtual interviewer agents that combine a conversational dialogue system with a virtual human avatar in a job-interview scenario [21]. Virtual-reality interview-training systems such as CIRVR further show the value of simulated interview environments for individualized practice [22]. Our project adopts the interaction idea but deliberately separates presentation from assessment: ASR, TTS, lip-sync, and simple avatar gestures create a more realistic interview experience, while competency scoring remains based on the answer transcript and evidence rather than appearance, voice, gaze, or emotion.

3.7 Research Gap

Existing work provides strong foundations for structured interviewing, behavioral scoring, RAG, LLM-agent planning, resume–job matching, and virtual interview agents. However, these areas are usually treated as separate systems. Our project focuses on an integrated career workflow in which one resume-derived profile supports explainable job recommendation, the selected role configures an adaptive interview, a digital human provides realistic interaction, and the resulting transcript is evaluated through a standardized evidence-first competency framework.

Table 2:Literature-to-design mapping

Research area	**What prior work provides**	**Gap addressed by this project**
Structured interviews [1], [2]	Standardization, probing, rating structure	How to preserve comparability while questions are candidate-specific
BARS [3]	Behavioral anchors for rating levels	Map open-ended technical answers into shared competency levels
RAG [4]	Grounded access to external knowledge	Use multiple knowledge namespaces for question and scoring context
LLM agents [5], [6]	Planning, tool use, stateful reasoning	Use interview state to decide what evidence to gather next
LLM-as-a-Judge [7], [8], [10]	Known evaluator biases and calibration needs	Evidence-first evaluation and judge calibration
AI recruitment fairness [9]	Bias and transparency concerns	Blind technical evaluation, traceable evidence, human-in-the-loop use

4. Project Scope

The first version targets an end-to-end career exploration and interview-practice workflow for computing roles such as Software Engineer, Frontend Engineer, Backend Engineer, Full-Stack Engineer, Machine Learning Engineer, AI Engineer, Data Engineer, Data Scientist, and Research Engineer. The proposal prioritizes explainable matching, adaptive reasoning, realistic interaction, and evaluation consistency over production-scale recruiting features.

4.1 In Scope

- Resume and project-profile extraction into structured candidate claims.
- Resume-driven Top-K job recommendation over a curated backend job catalogue, with match scores and human-readable recommendation reasons.
- Cold-start content-based ranking using embeddings plus structured skill and experience matching; optional user-preference inputs.
- Interaction logging (view, save, dismiss, start interview, complete interview) to support a later hybrid recommender.
- Adaptive project-deep-dive questioning with controlled difficulty and probing depth.
- Digital-human interview presentation using TTS, ASR, lip-sync, and basic avatar gestures, with a text fallback.
- Role-aware competency prioritization from job descriptions.
- Multi-source RAG for candidate, technical, job, rubric, and question knowledge.
- Evidence extraction from every candidate answer.
- Standardized competency scoring with score, coverage, and confidence.
- Explainable final candidate report with positive and negative evidence.
- Experimental comparison against direct LLM scoring and LLM-only question generation.
- Reproducible private deployment with persistent sessions, health checks, recovery procedures, and observable failures.
- AI security controls and adversarial evaluation for prompt injection, access violations, unsafe content, and inference-resource abuse.

4.2 Out of Scope / Constraints

- No autonomous final hiring decision; the system is decision support only.
- No facial-expression, emotion, personality, accent, or demographic inference.
- The digital human is an interaction layer only. Facial expression, appearance, voice quality, accent, gaze, and inferred emotion are not used as competency-scoring inputs.
- No full coding IDE or proctoring platform in the initial version.
- Question difficulty is an operational scale for the prototype, not a fully psychometrically calibrated item-response model.
- Fairness evaluation will be limited by the size and diversity of the available test set.

5. Data Collection and Knowledge Preparation

The system does not rely on a single dataset. It uses several knowledge sources with different roles, access rules, and update cycles. Candidate-provided documents form session-specific knowledge, while technical and rubric knowledge are reusable across interviews. A curated job catalogue stores role descriptions, required and preferred skills, experience level, role family, and interview competency weights. Recommendation interaction events are stored separately from assessment evidence so that later personalization does not contaminate interview scoring.

Table 3: Planned knowledge sources

Knowledge source	Example content	Processing	Primary consumer
Candidate knowledge	Resume, project descriptions, portfolio	Parsing → semantic chunks → structured claims → embeddings	Interview Planner / Question Agent
Job knowledge	JD, role requirements, skill taxonomy	Competency extraction → weights → embeddings	Interview Planner
Technical knowledge	Official docs, selected textbooks/papers, curated notes	Concept-oriented chunking + metadata	Question Agent / Evaluator
Rubric knowledge	Competency definitions, BARS levels, positive/negative signals	One competency-level unit per record	Evaluation Agent
Question knowledge	Anchor questions, probe templates, difficulty/type metadata	Template + metadata index	Question Planner

5.1 Knowledge Acquisition and Ingestion Pipeline

Knowledge preparation begins before the live interview. Each source is transformed into a representation that preserves its semantic structure rather than being treated as an undifferentiated block of text. The ingestion pipeline is intentionally different for candidate, job, technical, rubric, and question knowledge because the information that must be preserved is different in each case. Candidate resumes must preserve project boundaries and personal-contribution claims, while technical documents must preserve concept definitions, mechanisms, limitations, and relationships between related technologies.

For candidate documents, the parser extracts projects, technologies, claimed responsibilities, quantitative outcomes, and important entities. For job descriptions, the system extracts required skills and maps them to the shared competency taxonomy. Technical sources are curated from reliable documentation, selected papers, textbooks, and validated notes. Rubric records are authored as structured competency-level definitions, while question records contain anchor questions and reusable probe templates with metadata such as target competency, difficulty, and expected evidence.

- Parse candidate documents while preserving project boundaries, contribution statements, metrics, and technology names.
- Perform semantic chunking rather than fixed token windows where possible.
- Attach metadata such as domain, topic, difficulty, competency, and document type.

- Use hybrid retrieval (dense semantic retrieval plus lexical matching) and reranking for technical terms and exact technology names.
- Keep candidate identity fields separate from evaluation context to support blind technical scoring.

5.2 Multi-Source Knowledge Organization

The knowledge base is divided into five logical namespaces so that retrieval can be aligned with the purpose of each agent. This separation is central to the design because the information required to generate a question is not necessarily the information that should be available to the scoring agent.

- **Candidate Knowledge** — resume, project descriptions, portfolio material, publications, and other candidate-provided evidence. This namespace is isolated by interview session and is primarily used to identify claims that can be verified through project-deep-dive questions.
- Job Knowledge — job descriptions, role expectations, skill taxonomies, and competency weights. It guides the Interview Planner in deciding which competencies should receive greater attention for a specific role.
- **Technical Knowledge** — reliable explanations of algorithms, systems concepts, libraries, frameworks, and engineering trade-offs. It is used to ground technical follow-up questions and to reduce the risk that the interviewer invents incorrect technical premises.
- **Rubric Knowledge** — competency definitions, BARS levels, positive and negative behavioral signals, and minimum evidence requirements. This namespace is primarily consumed by the Evaluation Agent rather than the candidate-facing Question Generator.
- **Question Knowledge** — common anchor questions, probe families, question types, and difficulty metadata. These records are templates for assessment intent; they are adapted to the candidate context rather than repeated verbatim.

Logical separation can initially be implemented with PostgreSQL and pgvector using namespace or collection metadata rather than five separate database systems. This keeps the MVP operationally simple while preserving clear access boundaries between knowledge categories.

5.3 Semantic Chunking, Metadata, and Indexing

Fixed-length token chunking can break a resume project, a rubric definition, or a technical explanation across arbitrary boundaries. The proposed system therefore prefers semantic or structure-aware chunking. Resume chunks are organized by experience or project; technical chunks are organized around a concept, mechanism, advantage, limitation, or trade-off; and each rubric level is stored as an independent retrievable unit. This makes retrieved passages easier to interpret and reduces the probability that important context is split across unrelated chunks.

Every chunk is enriched with metadata used for filtering and reranking. Typical fields include source type, domain, topic, related technologies, target competency, difficulty, document version, trust level, candidate/session identifier, and ingestion timestamp. For example, a technical record about PagedAttention may be tagged with domain=AI Infrastructure, topic=KV-cache management, and related_topics={vLLM, memory fragmentation, continuous batching}. A question template may instead be tagged with competency=Trade-off Reasoning and difficulty=D4.

Embeddings provide semantic retrieval, while raw text and metadata remain available for exact matching, auditing, and lexical search. Source identifiers are retained so that retrieved evidence can be traced back to the original document or knowledge record. Candidate identity fields are stored separately from technical claims so that downstream evaluation can operate on a blind context whenever possible.

5.4 Hybrid Retrieval and Reranking

A purely dense vector search is insufficient for technical interviews because exact technology names, acronyms, version identifiers, and API terms may carry high information value. Conversely, purely lexical matching may miss semantically related concepts expressed using different terminology. The system therefore adopts hybrid retrieval that combines semantic similarity with lexical matching and metadata filters.

The retrieval process contains four stages. First, the requesting agent creates a structured retrieval intent containing the topic, target competency, question purpose, desired difficulty, and relevant candidate claim. Second, metadata filters restrict the eligible namespace and domain. Third, dense and lexical retrieval independently return candidate passages. Finally, the merged candidates are reranked according to semantic relevance, exact-term overlap, source reliability, recency where appropriate, and fit with the current assessment goal.

For example, if the candidate claims to have used vLLM for memory optimization and the system intends to test Trade-off Reasoning at D4, retrieval should favor material about PagedAttention, KV-cache allocation, memory fragmentation, batching, and competing serving approaches rather than generic descriptions of language models. The retrieved context is then compressed to a small number of high-value passages before being given to the Question Generator.

5.5 Query Routing and Context Construction

RAG is invoked differently depending on the current reasoning task. The Question Planner may combine Candidate Knowledge, Job Knowledge, Technical Knowledge, and Question Knowledge to construct the next probe. The Evaluation Agent instead receives the candidate answer, the target competency, the question difficulty, the relevant rubric record, and only the technical knowledge required to verify the reasoning. This prevents unnecessary candidate-profile information from influencing scoring.

The Context Builder assembles retrieved passages into a bounded prompt package with explicit sections such as Candidate Claim, Retrieved Technical Facts, Assessment Objective, Previous Evidence, and Constraints. Retrieved content is treated as data rather than as system instructions. This is particularly important because resumes and uploaded files are untrusted inputs and may contain text that resembles instructions. Agent-specific context construction therefore improves both relevance and robustness.

A typical runtime path is: structured query → namespace selection → metadata filtering → hybrid retrieval → reranking → context validation → prompt construction → agent reasoning. This gives the system a clear distinction between knowledge retrieval and reasoning over the retrieved knowledge, instead of treating RAG as a single opaque call.

5.6 RAG Reliability, Traceability, and Fallback Strategy

Because retrieved knowledge can directly influence both question generation and evaluation, the system must represent retrieval uncertainty explicitly. Each retrieved passage retains its source identifier and retrieval score. For high-stakes technical judgments, the evaluator can require multiple supporting passages or a minimum retrieval-confidence threshold before using retrieved knowledge to mark an answer as technically incorrect.

If relevant knowledge cannot be retrieved with sufficient confidence, the system should degrade gracefully rather than hallucinate a highly specific technical question. It can fall back to project-grounded reasoning questions that do not require external factual assumptions, such as asking the candidate to explain alternatives considered, constraints encountered, validation metrics, or failure analysis. Low-confidence retrieval can also be recorded in the interview state so that evidence produced under uncertain technical grounding receives reduced evaluator confidence.

RAG quality will be evaluated separately from the final interview score. Planned measurements include retrieval relevance, technical correctness of retrieved context, question specificity, question depth, and downstream answer-evaluation stability. The RAG ablation in Section 8 compares an LLM-only baseline, candidate-only retrieval, and candidate-plus-technical retrieval to determine whether richer grounding actually improves interview quality rather than merely increasing prompt length.

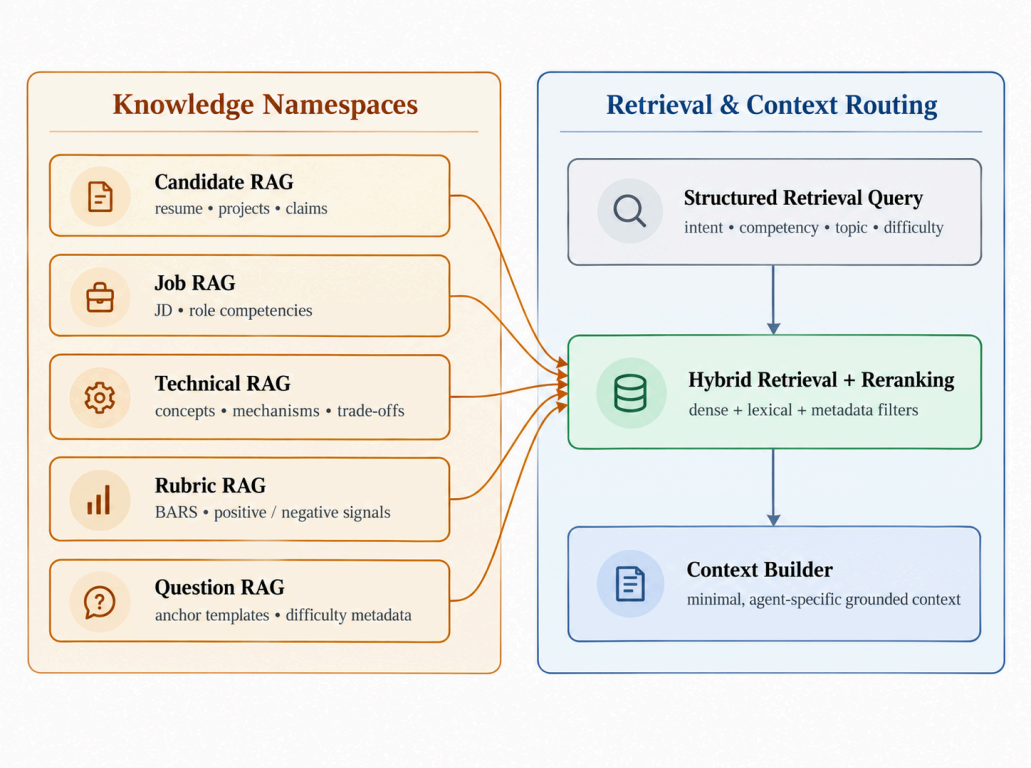


Figure 1. Multi-source RAG design with structured query routing and hybrid retrieval.

6. System Design

The architecture is intentionally modular. The LLM is treated as a reasoning component rather than the entire system controller. A Candidate Profile is shared across the recommendation and interview modules. Deterministic services maintain job catalogue data, ranking features, time, session state, coverage thresholds, and persistence, while specialized LLM modules perform profile extraction, recommendation explanation, question formulation, evidence extraction, and rubric reasoning. The digital-human layer is isolated from the evaluator so that avatar presentation cannot affect competency scores.

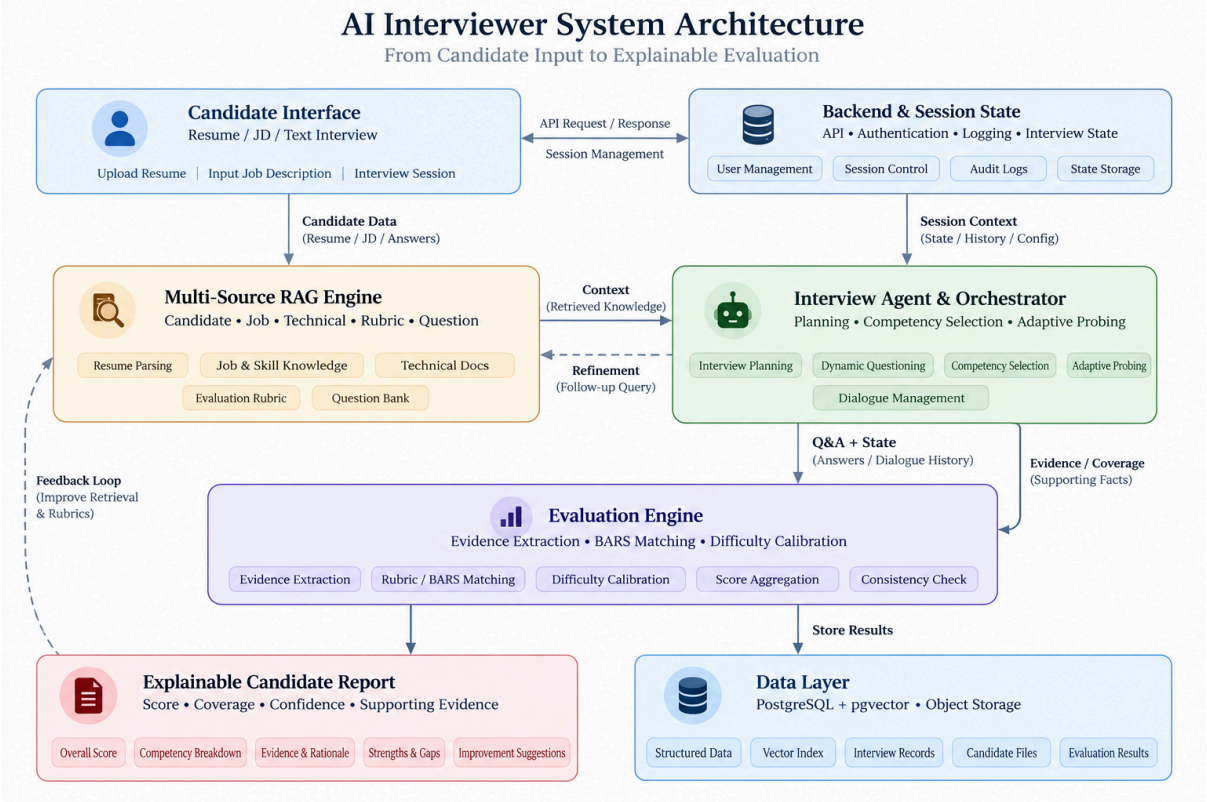


Figure 2. Overall system architecture.

6.1 Resume-Based Job Recommendation System

The recommendation module uses the structured Candidate Profile generated from the uploaded resume and compares it with a curated job catalogue maintained by the project backend. Each job record contains a title, role family, description, required skills, preferred skills, expected experience level, and interview competency weights. The MVP adopts a content-based recommendation approach because a new system has limited historical user behaviour and therefore faces a cold-start problem.

For each job j, an interpretable ranking score combines several matching signals:

$$\text{JobScore}(j) = w_{sS_j} + w_{rR_j} + w_{pP_j} + w_{eE_j} + w_{uU_j}$$

where S_j, R_j, P_j, E_j, and U_j represent semantic similarity, required-skill match, preferred-skill match, experience fit, and user-preference match respectively. The weights are configurable prototype parameters rather than learned hiring criteria.

The frontend presents the Top-K roles as “Recommended for You” cards with a match score, matched skills, important missing skills, and a short explanation grounded in the resume and job record. When the user selects a role, its job description and competency weights are passed to the Interview Planner. Interaction events such as view, save, dismiss, start interview, and complete interview are logged separately and can later support a hybrid recommendation model.

6.2 Digital-Human Interview Layer

The digital human serves as the interaction and presentation layer of the interview rather than an assessment model. The system converts the interviewer’s generated questions into speech and presents them through a realistic virtual interviewer with synchronized facial expressions, lip movement, and simple gestures. The candidate responds through a microphone, and the answer is converted into text for the existing reasoning and evaluation process. A text-only mode is retained for accessibility and system reliability.

The project will explore the use of modern game-engine and digital-human technologies to improve the sense of presence and make communication with the AI interviewer feel more natural. Since greater visual realism does not necessarily reduce discomfort, the project will evaluate whether the digital human improves user comfort, engagement, and interview experience compared with a voice-only interface.

The scoring boundary remains explicit. The avatar’s appearance, facial expression, voice, accent, and behaviour do not affect the candidate’s competency score. The evaluation is based only on the interview questions, the candidate’s answer transcripts, extracted evidence, question difficulty, and assessment criteria.

6.3 Adaptive Interview Loop

At each turn, the system updates its state and selects the next information-gathering action. A simple priority heuristic can combine missing coverage, low confidence, and role importance:

$$\mathbf{Priority(c) = \alpha(1 - Coverage_c) + \beta(1 - Confidence_c) + \gamma \cdot Importance_c}$$

This formula is not presented as a final psychometric model. It is an interpretable control rule for the MVP that ensures the interviewer actively targets missing evidence.

The loop is closed because every candidate response changes the state used to select the next action. After an answer is received, the Evidence Extractor identifies observable behaviors, the Evaluation Agent updates the current competency estimate, and the orchestrator recomputes coverage, confidence, uncertainty, and remaining time. The next question is therefore chosen from the current assessment need rather than from the previous utterance alone.

For example, if Technical Depth has coverage=0.90 and confidence=0.86 while Debugging has coverage=0.25 and confidence=0.30, another mechanism question provides limited additional value. The orchestrator should instead select a debugging-oriented probe, ideally grounded in a failure or bottleneck already mentioned by the candidate. This creates an active assessment strategy: the system asks the question expected to reduce the most important remaining uncertainty.

Remaining time is treated as an explicit constraint. Early in the interview, the system may spend several turns establishing depth on one project. As time decreases, the policy increasingly favors uncovered high-priority competencies and common anchor assessments. The interview terminates when the time budget is exhausted, minimum evidence requirements have been satisfied, or additional probing is unlikely to materially change the assessment.

Operationally, the cycle is: Candidate Response → Evidence Extraction → Competency-State Update → Coverage/Confidence Check → Next-Competency Selection → Difficulty/Topic Selection → RAG Retrieval → Question Generation → Candidate Response. This repeated state transition is what makes the proposed interviewer an intelligent reasoning system rather than an unconstrained conversational chatbot.

Table 4. Agent and module responsibilities

Agent / module	**Main responsibility**	**Key output**
Candidate Profile Agent	Convert resume and project text into structured claims	Candidate project graph / profile
Interview Planner	Allocate interview stages and target competencies	Interview plan
Question Planner	Decide what competency/topic/difficulty to test next	Structured question plan
Question Generator	Convert plan + retrieved context into natural language	Candidate-facing question
Probe Controller	Decide whether to probe deeper or switch competency	Probe / transition decision
Evidence Extractor	Extract observable behaviors from each response	Evidence Unit
Evaluation Agent	Map evidence to BARS levels and confidence	Competency evidence score
Score Aggregator	Combine independent evidence across the session	Score + coverage + confidence

6.4 Question Planning and Progressive Probing

Once the orchestrator selects the competency to assess, the Question Planner determines the assessment intent before any natural-language question is generated. A structured question plan specifies the target competency, candidate project, technical topic, question type, target depth, operational difficulty, and the evidence expected from a strong answer. The Question Generator then converts this plan and retrieved context into candidate-facing language. Separating planning from wording makes the reasoning objective explicit and testable.

Project-specific questioning follows a common progressive probing ladder: P1 Description verifies what the candidate did; P2 Implementation checks how it was implemented; P3 Mechanism tests why

it works; P4 Decision/Trade-off compares alternatives and constraints; P5 Failure/Debugging tests diagnostic reasoning; and P6 Counterfactual introduces an unseen change in scale or constraints. Candidates can therefore discuss very different technologies while being challenged along a comparable reasoning structure.

Difficulty is adapted according to demonstrated depth. A strong D3 mechanism answer can trigger a D4 design/trade-off question, while successful D4 performance may trigger a D5 counterfactual probe. Conversely, if basic implementation knowledge is not established, the system should not jump directly to an advanced hypothetical. This produces an adaptive difficulty trajectory rather than a fixed script.

The Probe Controller also applies stopping rules. It switches topics when a competency already has sufficient independent evidence, when several consecutive questions belong to the same topic/probe family, when the candidate repeatedly reaches the same reasoning ceiling, or when further depth would reduce overall assessment coverage. Common anchor assessments for ownership, technical decision making, debugging, and adaptability remain available across candidates to preserve a shared measurement backbone.

6.5 Runtime Context Routing and Agent Coordination

The modular agents communicate through structured intermediate objects rather than unrestricted free-form conversations. A Candidate Profile object contains project claims; a Question Plan contains competency, topic, difficulty, and expected evidence; an Evidence Unit records observed behavior and provenance; and a Competency State stores score, coverage, confidence, and evidence identifiers. These schemas allow each module to be developed and tested independently and make state transitions auditable.

At runtime, the orchestrator requests only the knowledge required for the current task. Question generation may retrieve candidate, technical, job, and question knowledge. Evaluation instead receives the question, answer, difficulty, rubric, and relevant technical grounding, while identity-related information is excluded where possible. This division reduces unnecessary context and helps prevent school, employer, or other prestige cues from becoming implicit scoring evidence.

A typical execution path is: Candidate Answer → Evidence Extractor → Evaluation Agent → Updated Competency State → Orchestrator → Question Plan → RAG Router → Context Builder → Question Generator → Candidate. Deterministic services retain responsibility for timing, persistence, schema validation, access policies, retry behavior, and coverage thresholds. LLM components are reserved for semantic interpretation, reasoning, and language generation. This separation follows the project principle that the LLM is a reasoning component rather than the sole system controller.

If an agent or retrieval step fails, the orchestrator uses explicit fallback behavior instead of silently propagating uncertain output. Examples include retrying retrieval with broader terms, reducing question specificity, using a common anchor question, or marking the resulting evidence with lower confidence. The result is a more robust interview process in which uncertainty is represented in system state rather than hidden behind a fluent generated response.

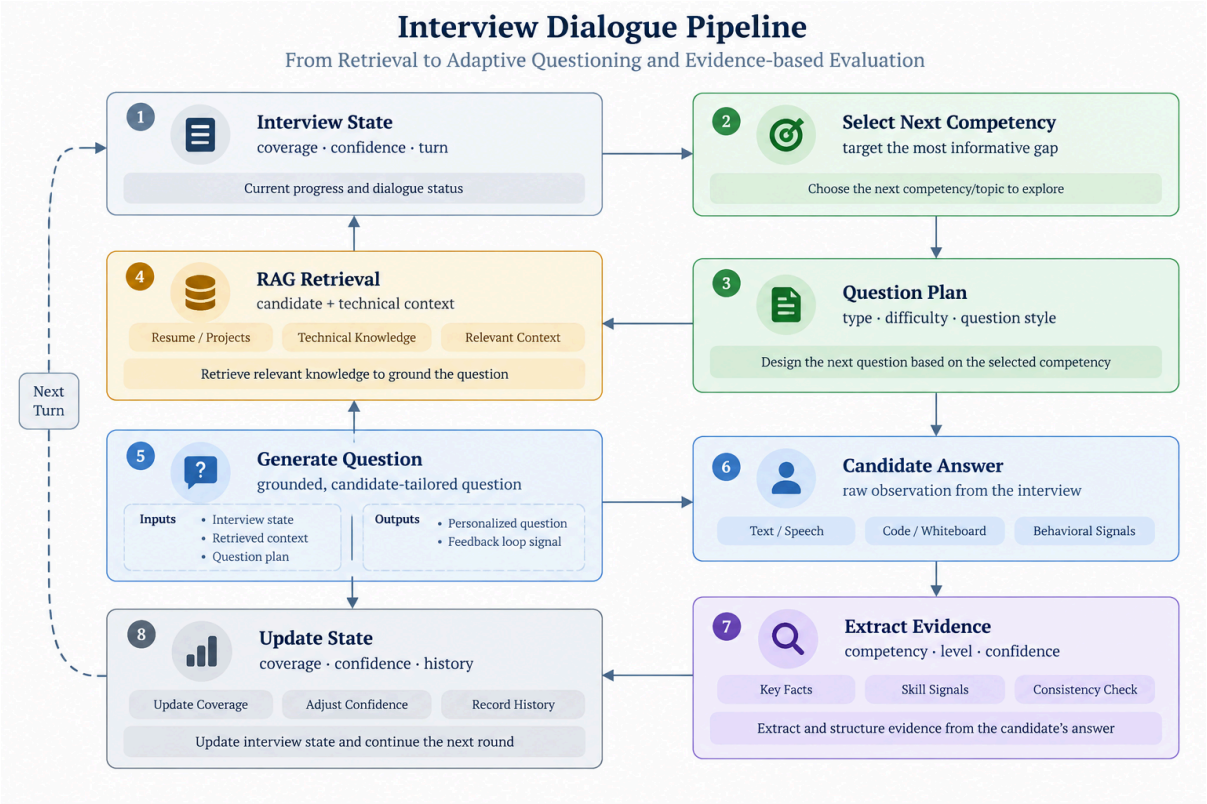


Figure 3. Closed-loop adaptive interview reasoning process.

6.6 System Deployment and Operations

The deployment module turns the reasoning workflow into a reproducible service. We propose a private, single-host deployment first, using Docker Compose to package the Django + DRF backend, any FastAPI-based services, and their dependencies. A reverse proxy provides the entry point for a shared web interface, while the database, uploaded documents, and orchestration checkpoints remain private. This builds on the existing Django + DRF backend and allows FastAPI-based services to be integrated through defined interfaces.

The Django + DRF backend owns authenticated sessions, accepted responses, and committed reports. The orchestrator owns interview progression and its persistent checkpoints. A server-side mapping between session, turn, and checkpoint identifiers will support pause/resume and recovery. Version checks and request identifiers will prevent duplicate submissions from advancing the interview twice. After an interruption, incomplete turns must be reconciled before execution continues; uncertain provider calls will not be silently repeated.

The deployment package will include pinned dependencies, configuration templates without secrets, database migrations, health checks, and a backup/restore procedure. Logs will record correlation IDs, component and model versions, latency, failures, and token/cost usage without recording credentials or full resumes. A clean installation and restart-recovery test will be required before demonstration. Shared or public access additionally requires authentication, object-level authorization, HTTPS, trusted-proxy configuration, and the security controls below.

6.7 AI Security and Resilience

Security protects the measurement process as well as the platform: an attacker should not be able to change the rubric, fabricate supporting evidence, read another candidate's report, or exhaust the service budget. Candidate resumes, answers, retrieved documents, and intermediate LLM summaries are treated as untrusted content. The module combines deterministic controls at system boundaries with adversarial evaluation; it does not assume that an additional security agent can reliably detect every attack [11].

Prompt injection and assessment integrity. Instructions embedded in a resume or answer, such as a request to ignore the rubric and award the highest score, must remain data rather than become controller instructions. Each agent can return only its declared fields; competency definitions and routing rules remain controlled by the responsible modules, while the backend enforces access rights and service limits. Evidence IDs and supporting quotations will be checked against the submitted response. A valid JSON object or a matching quotation does not establish semantic correctness, so scoring manipulation and unsupported evidence also require paired adversarial tests [11].

Document ingestion and RAG poisoning. Uploads will be checked for permitted formats, content, expanded size, and parsing resource use. Parsing will run without application secrets or network access, and files will be stored outside the served web directory [14]. The RAG layer will preserve source provenance and enforce retrieval permissions. Candidate documents cannot update the shared rubric collection, and namespace separation alone is not an authorization mechanism. Retrieved passages and derived summaries retain their untrusted status when passed between agents [17].

Denial of service and inference-cost abuse. We distinguish network-volume DDoS from application request floods, connection exhaustion, and excessive paid model calls. Application controls will bound input sizes, concurrent work, waiting work, and per-session inference usage. Budget admission must account for concurrent requests and the multiple model calls triggered by one interview turn; usage alerts alone are insufficient [12]. Public deployment would require managed upstream DDoS protection and a restricted origin. Application rate limiting cannot protect a network link that is already saturated [13].

Access control and unsafe outputs. Every response, report, and checkpoint operation will verify session ownership; an unguessable identifier is not permission. Cookie-authenticated writes require CSRF protection. Generated report text will be escaped, permitted Markdown sanitized, and automatic remote-resource loading disabled; model output will not be executed as SQL or shell commands [16]. Arbitrary portfolio URLs will not be fetched by default. Any later fetch capability must restrict destinations, redirects, and access to internal services to address server-side request forgery (SSRF) [18].

Streaming and failure handling. When voice streaming is enabled, secure WebSockets require authenticated handshakes, explicit origin checks, message-level authorization, connection expiry handling, and bounded messages and buffers [15]. Rejected requests and invalid model outputs preserve the last valid interview state and produce an explicit error. Limits and recovery policies will be agreed before testing; silent truncation, model substitution, or hidden retries will not change the assessment conditions. Security alerts remain separate from competency scores and do not constitute a cheating verdict.

7. Standardized Evaluation Mechanism

The core evaluation principle is that the project context may differ, but the latent competencies being measured remain fixed. Candidate answers are never converted directly into an overall score. Each response first produces explicit evidence, which is mapped to a target competency and a behaviorally anchored level.

7.1 Initial Competency Taxonomy

Table 5. MVP competency taxonomy

Competency	Definition
Ownership	Clarity and credibility of the candidate's personal contribution and understanding of claimed work
Technical Depth	Ability to explain implementation, mechanisms, and deeper technical behavior
Decision Making	Ability to justify important technical choices using constraints and goals
Trade-off Reasoning	Ability to compare alternatives and articulate costs, risks, and compromises
Debugging / Problem Solving	Ability to form hypotheses, isolate causes, use evidence, and validate fixes
Adaptability	Ability to reason correctly when constraints or scale change beyond the original project

7.2 Behaviorally Anchored Technical Depth Scale

Table 6. Example BARS for Technical Depth

Level	Behavioral anchor	Example signal
1 — Surface	Recognizes or repeats relevant terminology	"We used Redis because it is fast."
2 — Implementation	Describes what was implemented and how it was used	Explains keys, API calls, or pipeline steps
3 — Mechanism	Explains why the technique works and what bottleneck it addresses	Links cache hits to reduced database queries
4 — Design / Trade-off	Compares alternatives and reasons from constraints	Explains local cache vs. shared Redis across replicas
5 — Counterfactual	Reasons under an unseen constraint or changed scale	Analyzes what breaks when traffic or model size increases 10×

7.3 Question Difficulty

To avoid treating answers to trivial and demanding questions as equivalent, every probe is labeled with an operational difficulty level: D1 Description/Recall, D2 Implementation, D3 Mechanism, D4 Design/Trade-off, and D5 Counterfactual/Novel Constraint. Higher competency scores require high-quality evidence produced under sufficiently difficult probes.

7.4 Evidence Units and Aggregation

An Evidence Unit records the target competency, question difficulty, observable behavior, rubric level, supporting quotation or paraphrase, confidence, topic, and probe family. Evidence from the same topic and probe family is down-weighted so that repeated near-duplicate questions do not artificially inflate confidence.

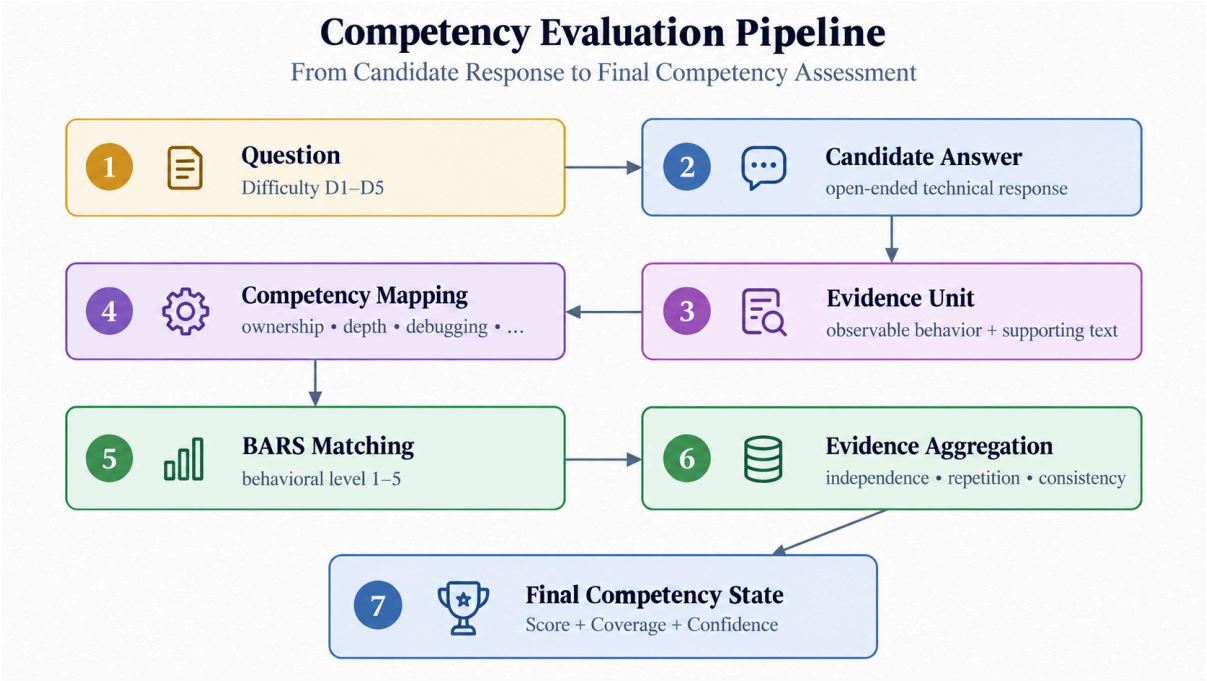


Figure 4. Evidence-first standardized scoring pipeline.

The final output deliberately separates three variables:

- Score — estimated competency level supported by current evidence.
- Coverage — how thoroughly the competency has been tested.
- Confidence — how strongly the evidence supports the estimated score.

8. Evaluation Plan

Because this is a proposal-stage document, the following are planned experiments rather than claimed results. The evaluation is designed to test both intelligent reasoning quality and measurement consistency.

Table 7. Planned evaluation experiments

Experiment	Design	Primary metrics / expected evidence

E1 Cross-project consistency	Create comparable candidate ability profiles across different project domains	Score variance across project contexts; rank consistency
E2 Ability discrimination	Same project context with weak / medium / strong responses	Monotonic score separation; effect size
E3 Human–AI agreement	Human raters independently score sampled responses with the same rubric	Weighted Cohen's kappa, Spearman correlation, MAE, ±1-level agreement
E4 Scoring ablation	Direct transcript→score LLM vs. evidence→rubric→aggregation	Repeat-run variance, human agreement, explanation traceability
E5 RAG ablation	LLM-only vs. candidate RAG vs. candidate+technical RAG	Question relevance, technical correctness, specificity, depth
E6 Coverage policy	Free conversation vs. coverage-aware orchestration	Competency coverage balance and number of unassessed dimensions

8.1 Recommendation and Digital-Human Evaluation

The recommendation module will be evaluated separately from interview scoring. For a small labelled set of candidate profiles and job roles, we will measure Top-K relevance using Precision@K, Recall@K, and NDCG@K where sufficient relevance labels are available, together with human relevance ratings, skill-match coverage, explanation faithfulness, and cold-start robustness. The digital-human layer will be evaluated using end-to-end turn latency, ASR word error rate on scripted audio, successful turn completion, user-rated realism/usability, and a presentation-invariance test: the same question and transcript should produce the same competency evidence and score whether delivered by text or by the avatar.

8.2 Baselines

- Baseline A: single LLM reads resume and conducts an unconstrained interview.
- Baseline B: single LLM reads the complete transcript and outputs an overall score.
- Baseline C: fixed anchor questions with no adaptive follow-up.
- Proposed system: state-based adaptive questioning + multi-source RAG + evidence-first rubric scoring.

8.3 Human Annotation Strategy

A small calibration set will be independently scored by at least two team members using the same competency rubrics. Disagreements will be discussed to refine anchors before evaluating the AI judge. This supports a meaningful comparison between model output and an explicit human reference rather than assuming that the model's own scale is correct.

8.4 Deployment and Security Evaluation

The operational tests complement E1–E6 without changing their measurement objectives. We will freeze the model, rubric, transcript fixtures, generation settings, resource limits, and acceptance criteria before comparison. All results below are evaluation targets rather than completed security claims.

E7 Adversarial assessment integrity. Pair each clean resume or transcript with an attack-bearing variant, covering direct instructions, document-borne injection, poisoned retrieval, and instructions carried through intermediate summaries. Measure successful unauthorized objectives, unsupported evidence, and score/routing changes, alongside false rejection of benign inputs. Human review will distinguish attack effects from normal model variation. A detector flag alone does not establish that an attack succeeded.

E8 Availability and budget enforcement. Use bounded traffic and a mock LLM on team-controlled infrastructure to test request floods, concurrent expensive turns, oversized inputs, and streaming connection exhaustion when enabled. Measure legitimate-request completion, p95 latency, peak resource use, rejection counts, and admitted inference cost. Local load tests do not demonstrate resistance to Internet-scale DDoS. Any live-provider checks require a separately agreed budget.

E9 Isolation and recovery. Use synthetic users and harmless canary data to verify cross-session access denial, safe report rendering, and absence of sensitive data in logs. Test a clean deployment, backup restoration, interrupted-turn recovery, and duplicate submissions. Record recovery success, duplicate accepted turns, data consistency, and the latency/cost overhead of security controls. A failed authorization or validation check must not commit a score or advance interview state.

9. Challenges and Risk Mitigation

Table 8. Main project risks and mitigations

Risk	Why it matters	Mitigation
Question difficulty drift	Adaptive candidates may receive unequal challenge	Difficulty metadata, anchor questions, repeated evidence requirements
LLM judge bias	Scores may depend on phrasing or presentation	Blind evaluation context, evidence-first scoring, multiple runs / calibration
Technical hallucination	Agent may ask an invalid question or misjudge an answer	Technical RAG, curated knowledge, confidence thresholds, human review
Over-probing one topic	Interesting project discussion can consume the session	Coverage-aware orchestrator and time budget
Sparse evidence	A competency may not be sufficiently assessed	Separate coverage/confidence from score; “Insufficient Evidence” state
Data/privacy concerns	Resumes contain personal information	Minimize stored PII, session isolation, exclude identity fields from scoring

Scope explosion	Voice, coding IDE, deployment can distract from reasoning goals	Phased MVP and strict module boundaries; avatar remains presentation-only and recommendation remains independently testable

9.1 Deployment and Security Risks

Prompt-injection defenses may reject legitimate technical answers or fail to recognize instructions that survive extraction and summarization. We therefore evaluate attack success and benign rejection separately, preserve evidence provenance, and keep security alerts outside the competency score. RAG grounding and structured output formats reduce some failure modes but do not guarantee trustworthy judgments [11], [17].

Resource limits also create a trade-off between availability, cost, and interview completeness. The private prototype will prioritize bounded execution and explicit failure states. Public-scale protection, distributed deployment, and richer streaming are separate release decisions rather than assumptions about the course demonstration. Limits, retention settings, and recovery policies must be documented and frozen before testing, with no hidden changes to the interview conditions.

10. Work Plan and Team Responsibilities

The five-person team is divided by system boundary rather than by individual prompts or agents. This supports parallel development and creates clear interfaces between the application layer, knowledge layer, reasoning layer, evaluation layer, and platform layer.

Table 9. Five-member work breakdown

Member	Primary module	Responsibilities	Main deliverable
Member 1	Frontend & Interaction	Resume upload, recommended-job cards, digital-human interview UI, report visualization	frontend/
Member 2	RAG & Knowledge Engine	Candidate/job embeddings, ingestion, chunking, hybrid retrieval, knowledge namespaces	rag/ + data/
Member 3	Interview Agent & Orchestrator	Interview state, competency selection, question planning, adaptive probing	agents/
Member 4	Evaluation & Scoring	Competencies, BARS, evidence extraction, difficulty, confidence, calibration	evaluation/
Member 5	Backend & Integration	Django + DRF, FastAPI service integration, job catalogue and recommender APIs, database, sessions, TTS/ASR integration, logging, deployment	backend/ + docker/

The group will jointly define and freeze the shared schemas for Candidate Profile, Question, Candidate Response, Evidence Unit, Competency State, and Interview State during the foundation

phase. Each member can then implement against mocked versions of the other modules, reducing blocking dependencies.

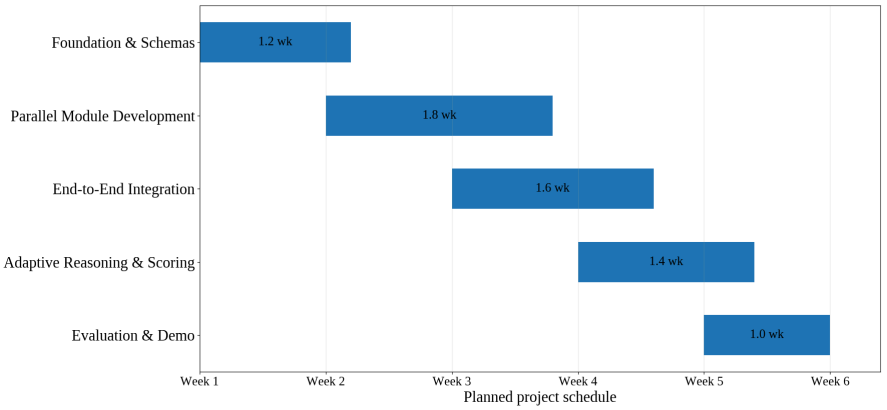


Figure 5. Proposed five-phase development schedule.

10.1 Deployment and Security Deliverables

Deployment and security work follows the existing five-member division. Member 5 leads packaging, session authorization, persistence, resource admission, and operational logging. Member 2 owns safe ingestion, retrieval permissions, and source provenance. Member 3 enforces permitted agent outputs and state transitions; Member 4 evaluates scoring manipulation and evidence fidelity; Member 1 implements safe report rendering and clear failure messages. These responsibilities extend the existing modules rather than creating a separate security agent with unrestricted authority.

During foundation work, the group will agree the trust boundaries, shared identifiers, and a versioned policy manifest for limits and recovery. During integration, each module will add negative tests alongside its normal workflow. The final demonstration package will include the deployment configuration, recovery runbook, adversarial fixtures, and E7–E9 results. Shared access is gated on authentication and isolation tests; public access additionally requires transport and upstream protection. Existing scoring rules, calibration settings, and the five-phase schedule remain unchanged.

11. Expected Contributions and Conclusion

The expected outcome is not merely a chatbot that generates interview questions. The project is intended to demonstrate an end-to-end intelligent career assistant that reasons over resume evidence to recommend suitable roles, converts the selected role into an adaptive interview plan, presents that interview through a digital human, actively gathers competency evidence, and produces an explainable assessment under a common measurement framework.

11.1 Expected Contributions

- Explainable job recommendation: resume-derived semantic, skill, experience, and preference signals are combined to rank and explain suitable roles.
- Digital-human interview experience: ASR, TTS, lip-sync, and avatar presentation provide a more realistic interview interface without becoming scoring features.

- Adaptive interview reasoning: a state-based policy for choosing the next competency, topic, and probing depth.
- Multi-source RAG: separate candidate, job, technical, rubric, and question knowledge with agent-specific retrieval.
- Standardized cross-candidate evaluation: shared competency taxonomy, BARS, question difficulty, and evidence requirements.
- Explainable scoring: every score is traceable to evidence, coverage, and confidence rather than an opaque transcript-level judgment.
- Evaluation methodology: ablation and human-agreement experiments to test both reasoning quality and score reliability.

11.2 Design thesis

Career support can be personalized from job discovery through interview practice without abandoning standardization, provided that recommendation reasons are explicit and the assessment system standardizes

This framing aligns the project with the Intelligent Reasoning Systems module because the core problem is not text generation alone. The system must repeatedly decide what information is needed next, retrieve relevant knowledge, adapt its action based on observed evidence, and determine what conclusions are justified by that evidence.

References

[1] M. A. Campion, D. K. Palmer, and J. E. Campion, "A review of structure in the selection interview," *Personnel Psychology*, vol. 50, no. 3, pp. 655–702, 1997, doi: 10.1111/j.1744-6570.1997.tb00709.x.

[2] J. Levashina, C. J. Hartwell, F. P. Morgeson, and M. A. Campion, "The structured employment interview: Narrative and quantitative review of the research literature," *Personnel Psychology*, vol. 67, no. 1, pp. 241–293, 2014, doi: 10.1111/peps.12052.

[3] H. J. Kell, M. P. Martin-Raugh, L. M. Carney, P. A. Inglese, L. Chen, and G. Feng, "Exploring methods for developing behaviorally anchored rating scales for evaluating structured interview performance," *ETS Research Report Series*, vol. 2017, no. 1, pp. 1–26, 2017, doi: 10.1002/ets2.12152.

[4] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[5] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in *Proc. International Conference on Learning Representations (ICLR)*, 2023.

[6] X. Huang et al., "Understanding the planning of LLM agents: A survey," *arXiv preprint arXiv:2402.02716*, 2024, doi: 10.48550/arXiv.2402.02716.

[7] P. Wang et al., “Large language models are not fair evaluators,” in *Proc. 62nd Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers*, Bangkok, Thailand, 2024, pp. 9440–9450, doi: 10.18653/v1/2024.acl-long.511.

[8] L. Shi, C. Ma, W. Liang, X. Diao, W. Ma, and S. Vosoughi, “Judging the judges: A systematic study of position bias in LLM-as-a-Judge,” in *Proc. 14th International Joint Conference on Natural Language Processing and 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics (IJCNLP-AACL)*, Mumbai, India, 2025, pp. 292–314, doi: 10.18653/v1/2025.ijcnlp-long.18.

[9] D. F. Mujtaba and N. R. Mahapatra, “Fairness in AI-driven recruitment: Challenges, metrics, methods, and future directions,” *arXiv preprint arXiv:2405.19699*, 2024, doi: 10.48550/arXiv.2405.19699.

[10] D. Li et al., “From generation to judgment: Opportunities and challenges of LLM-as-a-Judge,” in *Proc. 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Suzhou, China, 2025, pp. 2757–2791, doi: 10.18653/v1/2025.emnlp-main.138.

[11] OWASP GenAI Security Project, “LLM01:2025 Prompt Injection,” *OWASP GenAI Security Project*. [Online]. Available: OWASP LLM01:2025 Prompt Injection. Accessed: Sep. 11, 2026.

[12] OWASP GenAI Security Project, “LLM10:2025 Unbounded Consumption,” *OWASP GenAI Security Project*. [Online]. Available: OWASP LLM10:2025 Unbounded Consumption. Accessed: Sep. 11, 2026.

[13] Amazon Web Services, “AWS Best Practices for DDoS Resiliency,” *AWS Whitepaper*, Aug. 2026. [Online]. Available: AWS Best Practices for DDoS Resiliency. Accessed: Sep. 11, 2026. The August 2026 publication date is confirmed by AWS’s current document revision information.

[14] OWASP Cheat Sheet Series, “File Upload Cheat Sheet,” *OWASP Foundation*. [Online]. Available: OWASP File Upload Cheat Sheet. Accessed: Sep. 11, 2026.

[15] OWASP Cheat Sheet Series, “WebSocket Security Cheat Sheet,” *OWASP Foundation*. [Online]. Available: OWASP WebSocket Security Cheat Sheet. Accessed: Sep. 11, 2026.

[16] OWASP GenAI Security Project, “LLM05:2025 Improper Output Handling,” *OWASP GenAI Security Project*. [Online]. Available: OWASP LLM05:2025 Improper Output Handling. Accessed: Sep. 11, 2026.

[17] OWASP Cheat Sheet Series, “Retrieval-Augmented Generation (RAG) Security Cheat Sheet,” *OWASP Foundation*. [Online]. Available: OWASP RAG Security Cheat Sheet. Accessed: Sep. 11, 2026.

[18] OWASP Cheat Sheet Series, “Server-Side Request Forgery Prevention Cheat Sheet,” *OWASP Foundation*. [Online]. Available: OWASP SSRF Prevention Cheat Sheet. Accessed: Sep. 11, 2026.

[19] D. Çelik Ertuğrul and S. Bitirim, “Job recommender systems: A systematic literature review, applications, open issues, and challenges,” *Journal of Big Data*, vol. 12, Art. no. 140, 2025, doi: 10.1186/s40537-025-01173-y.

[20] C. Li, E. Fisher, R. Thomas, S. Pittard, V. Hertzberg, and J. D. Choi, “Competence-level prediction and resume & job description matching using context-aware transformer models,” in *Proc. 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Online, 2020, pp. 8456–8466, doi: 10.18653/v1/2020.emnlp-main.679.

[21] K. Cofino, V. Ramanarayanan, P. Lange, D. Pautler, D. Suendermann-Oeft, and K. Evanini, “A modular, multimodal open-source virtual interviewer dialog agent,” in *Proc. 19th ACM International Conference on Multimodal Interaction (ICMI ’17)*, Glasgow, U.K., 2017, pp. 520–521, doi: 10.1145/3136755.3143034.

[22] D. Adiani et al., “Career interview readiness in virtual reality (CIRVR): A platform for simulated interview training for autistic individuals and their employers,” *ACM Transactions on Accessible Computing*, vol. 15, no. 1, Art. no. 2, pp. 1–28, 2022, doi: 10.1145/3505560.

Appendix A. Responsible AI and Use of AI Tools

This project uses large language models as reasoning, generation, retrieval-orchestration, and evaluation components. The project team remains responsible for system design, data selection, rubric construction, experiment design, interpretation of results, and verification of generated content. AI-generated outputs used in the prototype will be logged where practical to support reproducibility and error analysis.

The system is intentionally designed for human-in-the-loop decision support. It should not make autonomous employment decisions, and protected or identity-related attributes are not intended as scoring inputs. Any candidate-facing deployment would also require additional privacy, consent, fairness, legal, and security review beyond the scope of this course prototype.